HackMatrix 2026 — Master Problem Statement Specification

Title: AI-Powered Web Threat Detection & Digital Trust Intelligence

Domains: Artificial Intelligence · Cybersecurity · Web3

Build Duration: 48 Hours (Deadline: **10 November 2026, 8:00 PM IST**)

Team: APEX WARCODE (HM-6FC44F) — Somnath Pal (Leader), Sreya Roy, Sudhansu kumar, Sukriti Singh

1. Background

- The internet powers online banking, education, recruitment, e-commerce, digital payments, business communication, and countless other services, making individuals and organisations increasingly vulnerable to digital threats.
- Phishing websites, fraudulent payment links, fake recruitment portals, impersonated organisations, malicious redirects, deceptive advertisements, and credential-harvesting attacks are becoming increasingly sophisticated.
- A fraudulent website can closely resemble a legitimate banking portal, university website, government service, or corporate platform, tricking users into sharing passwords, financial information, personal details, or authentication credentials.
- Beyond conventional phishing, the internet's underlying trust infrastructure faces security challenges: digital certificates and Certificate Authorities (CAs) establish secure HTTPS connections and verify website identities, but weak cryptographic implementations, compromised CAs, and improperly managed certificate trust chains can undermine this trust.
- **The eMudhra TLS Certificate Incident:** Demonstrated how weaknesses in CA key generation could potentially allow fraudulent certificates to be created for website impersonation, highlighting the importance of Certificate Transparency (CT), cryptographic validation, and continuous monitoring of the web's trust infrastructure.
- At the application level, insecure configurations, exposed services, outdated software, and other web security weaknesses further increase the risk of exploitation.
- Existing security tools address these problems independently; the goal is to build an accessible platform unifying intelligent threat detection, website security analysis, certificate validation, and collaborative threat intelligence.

2. The Challenge

- **Core Question:** How might we build an intelligent digital security platform that helps individuals and organisations identify phishing websites, fraudulent links, online scams, website impersonation, and suspicious digital activity before they become victims?
- **Domain Triad:** Combine **Artificial Intelligence**, **Cybersecurity**, and **Web3** to investigate

suspicious digital content, analyse website security, identify potential threats, and provide evidence-based security assessments.

- **Supported Inputs:** Anyone should be able to submit a suspicious **URL, website, domain, email, or message** and receive an understandable assessment of its potential risks.
- **Evidence-First Approach:** Rather than simply labelling websites as safe or malicious, the system must identify suspicious indicators, explain its findings, validate available security information, and help users make informed decisions.

3. Real-World Problem Scenarios

A. Phishing Websites and Suspicious Links

- **Common Attack Examples:**
  - Fake banking and payment gateways.
  - University admission and examination portals.
  - Government service and tax-payment websites.
  - Fake parcel delivery and payment confirmation links.
  - Fraudulent job application and internship portals.
  - Impersonated social media login pages.
  - Fake customer support and account recovery websites.
  - Malicious shortened URLs and redirect chains.
- **Indicators the Platform Must Identify:**
  - Domain spoofing and typosquatting.
  - Homograph attacks and deceptive character substitutions.
  - Suspicious URL structures and embedded redirect parameters.
  - Domains associated with known phishing campaigns.
  - Mismatches between the claimed organisation and actual website identity.
  - Unexpected redirects to unrelated domains.
  - Previously reported malicious destinations.
- **Expected Outcome:** Help users understand whether a link is potentially dangerous before opening it or sharing sensitive information.

B. Website Certificates and Digital Trust Vulnerabilities

- **Core Premise:** HTTPS only provides an encrypted connection and certificate-based authentication; it does **not** guarantee that a website is legitimate or free from fraud.
- **What the Platform Must Investigate:**
  - Expired or invalid TLS certificates.
  - Hostname and certificate identity mismatches.
  - Untrusted or revoked Certificate Authorities.
  - Suspicious or unexpected certificate issuances.
  - Weak cryptographic algorithms and inadequate key sizes.
  - Incorrect certificate chain configurations.
  - Certificate Transparency (CT) anomalies.
  - Known compromised Certificate Authorities and intermediate certificates.
- **Key Guardrail (eMudhra Scenario):**

    - The system may monitor Certificate Transparency information to identify unexpected certificates issued for particular domains.
    - It must distinguish between ordinary certificate configuration problems and security incidents involving compromised cryptographic trust.
    - In an eMudhra-like scenario, the platform should identify certificates linked to known compromised or distrusted CAs using trusted security intelligence and certificate chain information.
    - **Crucial Constraint:** *It must NOT claim to independently detect the internal key-generation weakness of a Certificate Authority merely from a website URL.*

C. Online Scams and Social Engineering

- **Common Social Engineering Vectors:**
    - Fake job and internship offers.
    - Investment and cryptocurrency scams.
    - Fake giveaways and prize notifications.
    - Fraudulent refund and cashback messages.
    - Impersonation of banks, universities, and companies.
    - Fake customer support communications.
    - Requests for OTPs, passwords, or payment credentials.
    - Fraudulent KYC and account verification notifications.
    - Fake technical support and remote-access scams.
- **Platform Capability:** Accept suspicious messages, email content, and communication text for analysis, identifying suspicious language, impersonation indicators, unusual payment requests, misleading links, and attempts to obtain sensitive information.

D. Website Security and Vulnerability Assessment

- **Safe, Non-Intrusive Security Checks:**
    - Missing or incorrectly configured security headers.
    - Insecure HTTP configurations.
    - Weak TLS configurations.
    - Exposed or publicly accessible sensitive endpoints where detectable through passive inspection.
    - Suspicious redirects and mixed-content issues.
    - Known software and dependency vulnerabilities where reliable version information is publicly available.
- **Requirement & Constraint:** Explain the potential impact of each finding and provide recommendations. **Do not** attempt to exploit vulnerabilities or perform intrusive scanning against websites without explicit authorisation.

4. Core Functional Requirements (6 Modules)

Module 1: Intelligent URL and Website Analysis

- **7 Inspection Steps:**
    1. Normalise and inspect submitted URLs.

2. Analyse domain structure and suspicious naming patterns.
3. Identify potential phishing and impersonation indicators.
4. Examine redirect chains and destination domains.
5. Check domain reputation against available threat intelligence sources.
6. Identify suspicious DNS and domain characteristics where available.
7. Associate detected indicators with known threat reports.

- **Required Outputs:** Website/domain info, identified suspicious indicators, known threat intelligence matches, redirect analysis, risk classification, supporting evidence, and recommended precautions.

Module 2: TLS Certificate and Web Trust Analysis

- **Checks Required:**
  - Certificate validity and expiration.
  - Domain identity and Subject Alternative Name (SAN) verification.
  - Issuing Certificate Authority and certificate chain.
  - Certificate revocation and known distrust information.
  - Public key strength and signature algorithms.
  - Certificate Transparency (CT) records.
  - Known compromised CA and intermediate certificate indicators.
- **Output & Rule:** Generate a certificate security report explaining whether the TLS configuration appears valid, suspicious, or potentially compromised based on available evidence. **A valid TLS certificate must never be treated as proof that the website itself is safe.**

Module 3: AI-Powered Scam and Message Analysis

- **Capabilities:**
  - Identify common social engineering patterns.
  - Detect urgency-based manipulation and suspicious payment requests.
  - Recognise impersonation attempts.
  - Extract URLs, email addresses, phone numbers, and other relevant indicators.
  - Correlate extracted indicators with available threat intelligence.
  - Explain potential risks in understandable language and recommend appropriate safety measures.
- **Constraint:** Distinguish between suspicious language and verified evidence of fraud. **A message must not be labelled fraudulent solely because it contains urgent or persuasive language.**

Module 4: Intelligent Threat Correlation

- **Goal:** Correlate related indicators across submitted reports to detect larger fraud campaigns (e.g., different phishing URLs claiming to be from the same bank that redirect to shared infrastructure or payment destinations).
- **Capabilities:**
  - Identify repeated domains and infrastructure.
  - Detect similarities between reported phishing messages.
  - Identify recurring impersonation patterns.

    - Correlate previously reported URLs and indicators.
    - Build relationships between potentially connected incidents.
- **Design Rule:** AI can assist in summarising relationships, while **deterministic matching and supporting evidence** must establish the actual connections.

Module 5: Website Security Assessment

- **Goal:** Inspect publicly available security information and report configuration weaknesses without attempting exploitation.
- **Each Finding Must Include:**
    - Identified security issue.
    - Technical evidence.
    - Potential impact.
    - Severity classification.
    - Recommended remediation.
    - Relevant limitations.
- **Constraint:** Any active testing must be restricted to explicitly authorised targets.

Module 6: Decentralized Threat Intelligence (Web3)

- **User Threat Reports Must Support:**
    - Suspicious URL or domain indicators.
    - Reported phishing campaigns.
    - Security incident summaries.
    - Cryptographic fingerprints (hashes) of reports.
    - Supporting evidence references.
- **Blockchain Integration Must Enable:**
    - Recording cryptographic commitments to security reports.
    - Maintaining a verifiable reporting history.
    - Independently checking report integrity.
    - Tracking report revisions, disputes, and validation status.
    - Establishing transparent provenance for submitted threat intelligence.
- **Privacy & Truth Rules:**
    - **Raw URLs, personal information, email content, passwords, and sensitive user data must remain OFF-CHAIN.**
    - **A blockchain record establishes the integrity and history of a report, NOT the truth of the report itself.**

5. Expected User Experience (5-Step Workflow)

1. **Step 1: Submit** — A user pastes a suspicious link, submits a domain, or provides a suspicious email or message.
2. **Step 2: Investigate** — The platform collects available domain information, certificate details, threat intelligence, and relevant technical indicators.
3. **Step 3: Analyse** — Cybersecurity checks identify suspicious patterns, while AI correlates the findings and generates an understandable explanation.
4. **Step 4: Assess Risk** — The platform provides a structured report describing the

identified threats, their supporting evidence, and recommended precautions.

5. **Step 5: Report and Verify** — Users can submit relevant incidents to the threat intelligence repository, where cryptographic commitments are recorded on-chain for independent verification.

6. Risk Classification Standard

Classification	Description
Low Risk	No significant malicious indicators identified within the available checks
Suspicious	One or more indicators require further investigation
High Risk	Multiple significant indicators suggest a substantial security threat
Confirmed Malicious	Supported by authoritative threat intelligence or sufficiently validated evidence
Inconclusive	Insufficient data or unavailable verification sources

- **Every Assessment Must Include:**
  - Overall risk classification.
  - Identified threat indicators.
  - Supporting technical evidence.
  - Source of each important finding.
  - Analysis timestamp.
  - Confidence or evidence coverage.
  - Recommended actions.
  - Limitations and unavailable checks.
- **Rule:** The system must not treat the absence of known threats as proof that a website is safe.

7. Mandatory 48-Hour MVP Requirements

Component	Minimum Requirement
URL Analysis	Accept and investigate submitted URLs or domains
Phishing Detection	Identify suspicious URLs and phishing indicators
TLS Analysis	Inspect HTTPS certificates and identify relevant trust issues
AI Integration	Generate evidence-grounded explanations of security findings
Scam Detection	Analyse pasted messages or email content
Risk Assessment	Produce structured and explainable risk classifications
Threat Intelligence	Integrate at least one external intelligence source or documented dataset

Component	Minimum Requirement
Web3 Integration	Implement an on-chain threat-report registry or equivalent
Verification	Independently verify a registered report's integrity
Dashboard	Provide a functional investigation interface
Testing	Demonstrate at least **three suspicious** and **three benign** test cases

8. Suggested Technology Stack

Domain	Possible Technologies
AI	LLMs, NLP, machine learning, anomaly detection, RAG
URL Intelligence	Google Safe Browsing, URLhaus, PhishTank, OpenPhish, VirusTotal
Web Security	Python SSL libraries, OpenSSL, DNS analysis, HTTP security-header inspection
Certificate Intelligence	Certificate Transparency logs, crt.sh, public CA trust information
Blockchain	Solidity, EVM-compatible networks, smart contracts
Backend	Python, FastAPI, Node.js
Frontend	React, Next.js, TypeScript
Database	PostgreSQL, MongoDB, Redis
Infrastructure	Docker, cloud hosting

(External APIs, open-source tools, existing models, and public security datasets are permitted; no need to train AI models from scratch.)

9. Security and Privacy Requirements

- **Safe URL Inspection:**
  - Protect against SSRF (Server-Side Request Forgery) and internal network access.
  - Validate redirects and destination IP addresses.
  - Avoid executing scripts or downloading potentially malicious files.
  - Use isolated environments for website inspection.
  - Apply request limits and timeouts.
- **AI Security:**
  - Treat webpage content and submitted messages as untrusted inputs.
  - Prevent prompt injection from influencing security decisions.
  - Require evidence for important findings.
  - Do not expose sensitive user information to external AI services unnecessarily.
- **Privacy:**
  - Never request passwords, OTPs, private keys, or seed phrases.
  - Avoid unnecessary collection of personal information.
  - Do not store sensitive content on public blockchains.
  - Protect API credentials and user-submitted reports.

- **Operational Safety:**
  - Do not execute malware or exploit vulnerable websites.
  - Do not perform intrusive security testing without authorisation.
  - Do not claim a website is secure simply because automated checks pass.

10. Expected Deliverables (9 Items)

1. **Working Prototype:** A functioning web application with URL or domain investigation capabilities.
2. **Public GitHub Repository:** Complete source code and setup instructions.
3. **AI Demonstration:** An example of intelligent scam analysis with evidence-based explanations.
4. **Cybersecurity Demonstration:** A phishing detection or suspicious website investigation, including TLS certificate analysis.
5. **Web3 Demonstration:** A deployed smart contract on a testnet or reproducible local blockchain setup.
6. **Verification Evidence:** Contract address and transaction reference or equivalent reproducible verification.
7. **Technical Documentation:** Architecture, analysis pipeline, threat intelligence sources, and security methodology.
8. **Testing Report:** Documented legitimate (at least 3) and suspicious (at least 3) test cases with observed results.
9. **Live Demonstration:** An end-to-end example showing submission, investigation, risk assessment, and threat reporting.

11. Evaluation Criteria

Category	Weight	Evaluation Focus
Cybersecurity & Threat Detection	**30%**	Phishing detection, certificate analysis, evidence quality, and false-positive handling
AI Implementation	**20%**	Intelligent investigation, contextual analysis, and explainability
Web3 Integration	**20%**	Meaningful threat reporting, integrity, and independent verification
Technical Implementation	**20%**	Working architecture, reliability, and integration quality
Innovation & User Experience	**10%**	Practical relevance, usability, and originality
Total	**100%**	—

Judges evaluate demonstrated functionality rather than the number of technologies or APIs integrated.

12. Optional Advanced Extensions

- Browser extension that checks links before navigation.
- Real-time phishing protection.
- QR code scam detection.
- Screenshot-based website impersonation detection.
- Email security analysis.
- Domain monitoring and alerts for suspicious certificate issuance.
- Detection of lookalike and homoglyph domains.
- Monitoring known compromised CA certificates.
- Automated security header and TLS configuration audits.
- Threat intelligence sharing APIs.
- Community-based scam reporting and validation.
- Organisation-level website security monitoring dashboard.

13. Scope Boundaries (What You Are NOT Expected to Do)

- Build a complete antivirus solution.
- Train foundation models from scratch.
- Detect every possible type of cyberattack.
- Reproduce or exploit real-world vulnerabilities.
- Build a complete commercial threat intelligence database.
- Support every Certificate Authority or blockchain network.
- Perform unauthorised vulnerability scanning.
- Guarantee that all websites are safe or malicious.

14. Final Objective

Build an accessible, intelligent digital security platform that helps individuals and organisations identify potentially malicious websites, avoid phishing attacks, recognise online scams, and understand the security risks associated with the websites they interact with. Demonstrate how **AI** investigates and correlates suspicious activity, **Cybersecurity** identifies and validates technical threats, and **Web3** establishes a transparent and verifiable mechanism for sharing threat intelligence.

15. Critical Winning Insights (From Hidden Constraints in Points 3–9)

1. **Strict Off-Chain Privacy Rule (Section 4, Module 6 & Section 9):**
    - Many teams will lose points by storing raw URLs or message text directly inside their Solidity smart contract. The brief explicitly mandates: *"Raw URLs, personal information, email content, passwords, and sensitive user data must remain off-chain."* Store only the **SHA-256 / Keccak-256 cryptographic commitment**

hash, timestamp, status (Reported / Verified / Disputed), and reporter address on-chain, while keeping the raw report in your off-chain DB and building a **"Verify Report Integrity"** button that hashes the off-chain JSON and compares it against the on-chain hash.

2. **Exact 5-Tier Risk Classification (Section 6):**
    - Use the exact 5 labels required by the rubric: **Low Risk**, **Suspicious**, **High Risk**, **Confirmed Malicious**, and **Inconclusive**. Never label a site "100% Safe."
3. **Mandatory 3 + 3 Test Case Suite (Section 7 & Section 10):**
    - Build a **"1-Click Demo Test Suite"** right on your dashboard with **3 Suspicious/Phishing samples** and **3 Benign samples** pre-loaded (plus custom input), and document their outputs in your README/Testing Report.
4. **SSRF & Prompt Injection Protection (Section 9):**
    - Add a visible security sanitizer in your backend (blocking 127.0.0.1, localhost, 169.254.169.254, private IP ranges 10.x.x.x/192.168.x.x before fetching any URL, and wrapping scraped text in untrusted delimiters before sending to the LLM) and mention this in your architecture docs.